

Desarrollo de API RESTful con CRUD y Capa de Servicios

Trabajo Práctico Final

Curso de Backend Developer

Objetivo General

Desarrollar un API RESTful robusta y modular utilizando Node.js, Express y MongoDB (Mongoose). El proyecto debe implementar las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y aplicar el patrón de separación de responsabilidades, delegando la lógica de negocio a una Capa de Servicios.

Consigna

Requisitos Funcionales y Técnicos

1. Entidades y Modelado

Entidad	Campos Clave	Requisitos CRUD
Productos	Nombre, Descripción, Precio, Stock, Categoría (Referencia)	Implementar todas las operaciones CRUD.
Categorías	Nombre, Descripción	Implementar principalmente GET, POST, DELETE.
Usuarios	Nombre/Email, Contraseña	Opcional: Requerido para la autenticación.

2. Estructura y Lógica de Negocio (Separación de Responsabilidades)

- Capa de Servicios (Obligatoria): Toda la lógica de negocio, manipulación de datos y llamadas directas a Mongoose/MongoDB deben residir en la carpeta `services/`.
- Controladores: Los controladores (controllers/) deben ser ligeros. Su única función es manejar la solicitud HTTP (Request), delegar la lógica al Servicio correspondiente y gestionar la respuesta HTTP (Response).
- Relaciones: La entidad Producto debe hacer referencia a Categoría. Utilice `populate` en las operaciones de lectura de productos para incluir los datos completos de la categoría.
- Manejo de Respuestas: Asegurar que los Controladores y Servicios manejen adecuadamente los mensajes de estado y códigos HTTP apropiados (ej. 200, 201, 400, 404, 500) y las excepciones (try/catch).

3. Seguridad y Configuración

- Encriptación: Utilizar `bcrypt` para hashear las contraseñas de los usuarios antes de guardarlas (mediante un *pre-save hook* de Mongoose).
- Autenticación con JWT (Opcional):
- Implementar una ruta de inicio de sesión que genere un JSON Web Token (JWT).
- Crear un middleware para verificar el JWT y proteger rutas sensibles (Creación, Actualización y Eliminación).
- Variables de Entorno: Usar `dotenv` para gestionar variables de configuración sensibles (URI de MongoDB, clave secreta de JWT).
- CORS: Habilitar el middleware CORS en la aplicación principal (`app.js`).

Estructura de Directorios Requerida

El proyecto debe mantener esta estructura para garantizar la separación de la arquitectura:

```
proyecto-crud-mongodb/  
├── app.js / index.js (Punto de entrada)  
├── .env  
├── package.json  
└── src/  
    ├── models/  
    │   ├── productModel.js  
    │   ├── userModel.js  
    │   └── categoryModel.js  
    ├── services/  
    │   ├── productService.js  <- Lógica de Negocio y DB  
    │   ├── userService.js  
    │   └── categoryService.js  
    ├── controllers/  
    │   ├── productController.js  <- Maneja HTTP  
    │   ├── userController.js  
    │   └── categoryController.js  
    ├── routes/  
    │   ├── productRoute.js  
    │   ├── userRoute.js  
    │   └── categoryRoute.js  
    ├── config/  
    │   └── db.js / conexion.js (Conexión a MongoDB)  
    └── middleware/  
        └── verifyToken.js      (Middleware JWT)
```

Detalle de la Entrega

El proyecto final debe ser entregado cumpliendo con los siguientes puntos:

1. Repositorio en GitHub: El código completo debe subirse a un repositorio público en GitHub.
2. Documentación (README.md): El repositorio debe incluir un archivo README.md detallado que contenga:

3. Título y una breve descripción del proyecto.
4. Esquema de la DB (Estructura de las colecciones).
5. Listado de Tecnologías utilizadas.
6. Instrucciones claras sobre cómo correr el proyecto (comandos de instalación y ejecución).
7. Listado completo de Endpoints (Rutas) disponibles (CRUD y Login).
8. Ejemplos de datos Mock (en formato JSON) para todas las solicitudes.
9. Funcionalidades implementadas, funcionalidades a futuro (si las hay)
10. Archivo .env: Incluir un archivo .env de ejemplo con la cadena de conexión a la base de datos (con credenciales de ejemplo o *placeholder*). Subir archivo .env en la entrega.

Aclaraciones

- Si por razones de seguridad no querés entregar el .env deberás hostear tu aplicación.
- No es obligatorio tener un front end.
- Se pueden utilizar colecciones distintas a las solicitadas, pero al menos un campo debe ser referenciado.
- Respecto a github si lo quieren poner privado nos deben enviar invitación a los docentes para ser colaboradores.